

```

# =====
# STEP 1: INSTALL DEPENDENCIES
# =====
!pip install -q transformers datasets evaluate sentencepiece sacrebleu rouge-score

# =====
# STEP 2: IMPORT LIBRARIES
# =====
import torch
import pandas as pd
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
from datasets import Dataset
from evaluate import load
from sklearn.model_selection import train_test_split

# =====
# STEP 3: DEVICE SETUP
# =====
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

# =====
# STEP 4: CREATE DATASET
# =====
data = {
    "question": [
        "What is a bank?", "What is ATM?", "How to open a bank account?",
        "What is minimum balance?", "What is a debit card?",
        "What is a credit card?", "How to check account balance?",
        "What is internet banking?", "What is IFSC code?",
        "How to transfer money online?", "What is a fixed deposit?",
        "What is interest rate?", "How to apply for loan?",
        "What is KYC?", "How to block ATM card?",
        "What is net banking password?", "What is mobile banking?",
        "What is overdraft?", "What is RTGS?", "What is NEFT?"
    ],
    "answer": [
        "A bank is a financial institution that manages money and provides loans.",
        "ATM stands for Automated Teller Machine used for withdrawing money.",
        "You can open a bank account by submitting ID proof and application form.",
        "Minimum balance is the least amount required to keep in your account.",
        "A debit card allows you to spend money directly from your bank account.",
        "A credit card allows you to borrow money up to a limit for

```

```

purchases.",
    "You can check balance using ATM, mobile app, or internet
banking.",
    "Internet banking allows you to perform transactions online.",
    "IFSC code identifies a bank branch for online transactions.",
    "You can transfer money using NEFT, RTGS, or mobile banking.",
    "A fixed deposit is an investment where money is locked for a
period.",
    "Interest rate is the percentage earned or paid on money.",
    "You can apply for a loan by submitting documents to the
bank.",
    "KYC means Know Your Customer, required for identity
verification.",
    "You can block your ATM card by calling customer care or via
app.",
    "It is a password used to access your online banking
account.",
    "Mobile banking allows transactions through a smartphone
app.",
    "Overdraft allows you to withdraw more than your balance.",
    "RTGS is a real-time fund transfer system for large amounts.",
    "NEFT is an electronic fund transfer system."
]
}

```

```
df = pd.DataFrame(data)
```

```
# =====
```

```
# STEP 5: TRAIN-TEST SPLIT
```

```
# =====
```

```

train_q, test_q, train_a, test_a = train_test_split(
    df['question'].tolist(),
    df['answer'].tolist(),
    test_size=0.3,
    random_state=42
)

```

```

train_dataset = Dataset.from_dict({"question": train_q, "answer":
train_a})

```

```

test_dataset = Dataset.from_dict({"question": test_q, "answer":
test_a})

```

```
# =====
```

```
# STEP 6: LOAD MODEL
```

```
# =====
```

```
model_name = "google/mt5-small"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForSeq2SeqLM.from_pretrained(model_name).to(device)
```

```

# =====
# STEP 7: TOKENIZATION (FIXED)
# =====
max_len = 128

def preprocess(batch):
    inputs = tokenizer(
        ["question: " + q for q in batch["question"]], # [] FIX
        padding="max_length",
        truncation=True,
        max_length=max_len
    )

    targets = tokenizer(
        batch["answer"],
        padding="max_length",
        truncation=True,
        max_length=max_len
    )

    labels = [
        [(t if t != tokenizer.pad_token_id else -100) for t in seq]
        for seq in targets["input_ids"]
    ]

    inputs["labels"] = labels
    return inputs

train_tokenized = train_dataset.map(preprocess, batched=True)
test_tokenized = test_dataset.map(preprocess, batched=True)

# [] FIXED FORMAT
train_tokenized.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "labels"]
)

test_tokenized.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "labels"]
)

# =====
# STEP 8: TRAINING
# =====
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-5)

model.train()
print("\n=== TRAINING STARTED ===")

```

```

for epoch in range(2):
    total_loss = 0

    for batch in train_tokenized:
        batch = {k: v.unsqueeze(0).to(device) for k, v in
batch.items()}

        optimizer.zero_grad()
        outputs = model(**batch)
        loss = outputs.loss

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}/2 | Avg Loss:
{total_loss/len(train_tokenized):.4f}")

# =====
# STEP 9: EVALUATION
# =====
model.eval()

rouge = load("rouge")
bleu = load("bleu")

predictions = []
references = []

with torch.no_grad():
    for batch in test_tokenized:
        input_ids = batch["input_ids"].unsqueeze(0).to(device)

        outputs = model.generate(
            input_ids,
            max_new_tokens=64,
            num_beams=4
        )

        pred = tokenizer.decode(outputs[0], skip_special_tokens=True)

        ref = tokenizer.decode(
            [x for x in batch["labels"].tolist() if x != -100],
            skip_special_tokens=True
        )

        predictions.append(pred)
        references.append(ref)

# =====

```

```

# STEP 10: METRICS (FIXED)
# =====
rouge_results = rouge.compute(
    predictions=predictions,
    references=references
)

bleu_results = bleu.compute(    # □ FIXED
    predictions=predictions,
    references=references
)

print("\n=== RESULTS ===")
print(f"ROUGE-L Score: {rouge_results['rougeL']:.4f}")
print(f"BLEU Score: {bleu_results['bleu']:.4f}")

# =====
# STEP 11: SAMPLE OUTPUTS
# =====
print("\n=== SAMPLE PREDICTIONS ===")

for i in range(min(3, len(predictions))):
    print("Q:", test_q[i])
    print("Pred:", predictions[i])
    print("Actual:", test_a[i])
    print("-" * 50)

```

Using device: cuda

```

{"model_id": "2cbd8a67162e4fde8ac5091204a488e9", "version_major": 2, "version_minor": 0}

```

The tied weights mapping and config for this model specifies to tie shared.weight to encoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning

The tied weights mapping and config for this model specifies to tie shared.weight to decoder.embed_tokens.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning

The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints, so we will NOT tie them. You should update the config with `tie\_word\_embeddings=False` to silence this warning

```

{"model_id": "ecdee887966445e9a62c43355cada311", "version_major": 2, "version_minor": 0}

```

```

{"model_id": "f8ba2219e5714861bca7dd3533525415", "version_major": 2, "version_minor": 0}

```

=== TRAINING STARTED ===

Epoch 1/2 | Avg Loss: 25.6678

Epoch 2/2 | Avg Loss: 30.5309

=== RESULTS ===

ROUGE-L Score: 0.0000

BLEU Score: 0.0000

=== SAMPLE PREDICTIONS ===

Q: What is a bank?

Pred: <extra_id_0>

Actual: A bank is a financial institution that manages money and provides loans.

Q: What is overdraft?

Pred: <extra_id_0>

Actual: Overdraft allows you to withdraw more than your balance.

Q: What is net banking password?

Pred: <extra_id_0>

Actual: It is a password used to access your online banking account.
